Examen parcial de teoría de SO

Instrucciones para el examen:

- **Justifica todas tus respuestas del examen.** Las respuestas no justificadas se considerarán erróneas.
- Podéis consultar el man instalado en el PC **pero ningún otro material.**
- **Utiliza el fichero template** que os damos junto con este enunciado para responder a las preguntas del examen y **entrégalo a través del racó** (solo un fichero).
- Utiliza un editor **en modo texto**.
- **Utiliza solo el espacio** que hay **reservado** para cada respuesta.
- Si no puedes responder una pregunta deja en blanco el espacio reservado para esa pregunta (no borres ninguna pregunta).

Pregunta Procesos (*5 puntos*)

Tenemos un ordenador que tiene una única unidad de ejecución y un Sistema Operativo Linux como el explicado en clase. Tenemos el siguiente código fuente correspondiente al programa "prog":

```
1.  void main(int argc, char **argv){
2.       char buf[512];
3.       printf("El proceso %d tiene el parametro de entrada %s\n", getpid(), argv[1]);
4.       sprintf(buf, "%d", atoi(argv[1])-1);
5.       if (atoi(buf) > 0) {
6.            if (fork() > 0)
7.                 execlp(argv[0], argv[0], buf, NULL);
8.       }
9.       func();         /* función que ejecuta código de forma activa durante 1 segundo */
10.      printf("El proceso %d finaliza con el valor %s\n", getpid(), argv[1]);
11.      exit(atoi(argv[1]);
12. }
```

figura 1 Código de prog

Contesta a las siguientes preguntas, argumentando brevemente tus respuestas, asumiendo que ejecutamos este código (donde todas las llamadas a sistema se ejecutan correctamente) con la siguiente línea de comandos: **./prog 4**

1) ***(0,5 puntos)*** Indica qué tipos de eventos han provocado ejecutar código del kernel durante la ejecución del programa, asumiendo que no ha existido ninguna interacción humana durante todo este tiempo.
RESPUESTA: Interrupciones del reloj (cada tick del SO, aprox 10ms) y llamadas a sistema.

2) ***(0,5 puntos)*** Dibuja la jerarquía de procesos que crea esta ejecución asignando un PID a cada proceso siguiendo la metodología indicada en las transparencias y en clase. Además, explica brevemente los pasos que han ejecutado los procesos para crear la jerarquía representada. Por último, explica brevemente si tenemos un esquema de creación y ejecución de procesos concurrente o secuencial.

RESPUESTA: La jerarquía de procesos es la siguiente:
P0
|----P0.0
|----P0.1
|----P0.2

El proceso P0 crea el primer hijo, entra en la condición del "if" y por lo tanto muta para volver a ejecutar el mismo binario, pero con un parámetro de entrada que tiene un número con una unidad inferior, y crea el siguiente hijo y así sucesivamente. Tiene un esquema de creación de procesos concurrente, ya que no existe ninguna llamada al sistema bloqueante, como waitpid, que fuerce un esquema secuencial.

3) ***(0,75 puntos)*** Indica los mensajes que se muestran por pantalla, explicando brevemente los pasos que han ejecutado los procesos para mostrarlos. Además, explica si podemos garantizar que los mensajes siempre aparecerán en el mismo orden o no.
RESPUESTA: Los mensajes son:
El proceso P0 tiene el parámetro de entrada 4
El proceso P0 tiene el parámetro de entrada 3
El proceso P0 tiene el parámetro de entrada 2
El proceso P0 tiene el parámetro de entrada 1
El proceso P0.0 finaliza con el valor 4
El proceso P0.1 finaliza con el valor 3
El proceso P0.2 finaliza con el valor 2
El proceso P0 finaliza con el valor 1
Los 4 primeros mensajes aparecen en este orden porque el proceso P0 ejecuta->escribe->crea proceso ->muta y vuelve a empezar. Los últimos 4 mensajes salen por la ejecución concurrente de todos los procesos. Por ello, los primeros 4 mensajes podemos garantizar su orden, pero no lo podemos garantizar en los últimos 4 mensajes, ya que todos los procesos están vivos y se ejecutan concurrentemente, excepto que por el orden de ejecución de los códigos y las llamadas a sistema, es altamente probable que el último sea la mayoría de las veces P0.

4) ***(0,75 puntos)*** Queremos añadir, entre las líneas de código 9 y 10, la llamada a sistema correspondiente que permita que un proceso espere la muerte de su proceso hijo (no hace falta obtener el valor de finalización), pero ha de tener un comportamiento no bloqueante. Indica la línea de código que añadirías y explica brevemente: (1) qué procesos libera cada proceso que ejecuta esta línea; y (2) de qué manera consideras que afectaría al orden de los mensajes mostrados por pantalla comparado con la respuesta del apartado anterior.
RESPUESTA: while(waitpid(-1, NULL, WNOHANG) >= 0);
Sólo el proceso P0 es el único que libera procesos (P0.0, P0.1 y P0.2). Los otros procesos, como no tienen hijos, no liberan ninguno y no iteran en el bucle. Por último, no afectaría al orden de los mensajes, porque el único proceso que tiene hijos es el P0. Por lo tanto, el orden de los últimos mensajes sigue siendo sensible a cambios de orden por la concurrencia, excepto que el último será siempre el mensaje de P0, forzado por el waitpid.

A partir del código original de la Figura 1, hacemos un cambio en la línea 6 que pasará a ser “if (fork() == 0)”. Contesta, asumiendo haber aplicado este cambio, a las siguientes preguntas:

5) ***(0,5 puntos)*** Dibuja la jerarquía de procesos que crea esta ejecución asignando un PID a cada proceso siguiendo la metodología indicada en les transparencias y en clase. Además, explica brevemente los pasos que han ejecutado los procesos para crear la jerarquía representada. Por último, explica brevemente si tenemos un esquema de creación y ejecución de procesos concurrente o secuencial.
RESPUESTA: La jerarquía de procesos es la siguiente:

```
P0
 |----P0.0
        |----P0.0.0
               |----P0.0.0.0
```

En este caso, el proceso que muta es cada proceso hijo. Por lo tanto, cada vez que muta, esta nueva ejecución creará un nuevo proceso hijo, creando esta jerarquía en forma de cascada. Tiene un esquema de creación de procesos concurrente, al igual que en el apartado "2", ya que no existe ninguna llamada al sistema bloqueante, como waitpid, que fuerce un esquema secuencial.

6) ***(0,75 puntos)*** Indica los mensajes que se muestran por pantalla, explicando brevemente los pasos que han ejecutado los procesos para mostrarlos. Además, explica si podemos garantizar que los mensajes siempre aparecerán en el mismo orden o no.
RESPUESTA: Los mensajes son:
El proceso P0 tiene el parámetro de entrada 4
El proceso P0.0 tiene el parámetro de entrada 3
El proceso P0.0.0 tiene el parámetro de entrada 2
El proceso P0.0.0.0 tiene el parámetro de entrada 1
El proceso P0 finaliza con el valor 4
El proceso P0.0 finaliza con el valor 3
El proceso P0.0.0 finaliza con el valor 2
El proceso P0.0.0.0 finaliza con el valor 1
Los 4 primeros mensajes aparecen en este orden porque el proceso P0 ejecuta->escribe->crea proceso -> hijo muta y vuelve a empezar. Aunque no existe ninguna llamada a sistema que fuerce el orden de los últimos 4 mensajes, aunque estén en ejecución concurrente, debido a las llamadas que hace cada proceso y el código que se ejecuta, es muy probable que siempre salgan en este orden.

7) ***(0,75 puntos)*** Asumiendo que se ha añadido correctamente la línea de código con la llamada a sistema del apartado “4”, explica brevemente en este nuevo caso qué procesos liberan a cada proceso ejecutando esta línea y de qué manera consideras que afectaría al orden de los mensajes mostrados por pantalla comparado con la respuesta del apartado anterior.
RESPUESTA: El procés P0 espera a P0.0. Aquest últim espera a P0.0.0. Aquest últim espera a P0.0.0.0 Aquest últim NO espera a ningú perquè no té fills.
Els missatges passaran a ser totalment deterministes, ja que ho forçarà la crida a sistema waitpid:
El procés P0 té el paràmetre d’entrada 4
El procés P0.0 té el paràmetre d’entrada 3
El procés P0.0.0 té el paràmetre d’entrada 2
El procés P0.0.0.0 té el paràmetre d’entrada 1
El procés P0.0.0.0 finalitza amb el valor 1
El procés P0.0.0 finalitza amb el valor 2
El procés P0.0 finalitza amb el valor 3

El procés P0 finalitza amb el valor 4

8) ***(0,5 puntos)*** ¿Cómo afectaría en el tiempo de ejecución completo del programa si tuviéramos un ordenador con más unidades de ejecución? Además, ¿afectaría a alguna de las respuestas anteriores de este ejercicio? Razona brevemente tu respuesta.
RESPUESTA: Tardaría menos tiempo porque más de un proceso podría estar ejecutando las instrucciones de la función "func" a la vez, así como el resto del código. Ahora bien, no afectaría en nada al comportamiento de la ejecución. Es decir, no afectaría a ninguna de las respuestas anteriores de este ejercicio.

Pregunta Signals (*5 puntos*)

Queremos hacer un programa que cree N procesos que ejecutaran los programas que nos pasan por línea de comandos y controlar el tiempo de ejecución de los procesos hijos. Tenemos el código mostrado en la figura 2 (está incompleto).

```
 1 void Usage()
 2 {
 3     char buffer[128];
 4     sprintf(buffer,"Usage: ejecuta duracion prog1 [prog2... progn]\n");
 5     write(2,buffer,strlen(buffer));
 6     exit(1);
 7 }
 8
 9 void f_handler(int s)
10 {
11     if (s == SIGALRM){
12     }
13     if (s == SIGCHLD){
14     }
15 }
16
17 int main(int argc,char *argv[])
18 {
19     if (argc < 3) Usage();
20     struct sigaction sa;
21     sa.sa_handler = f_handler;
22     sigemptyset(&sa.sa_mask);
23     sa.sa_flags = SA_RESTART;
24     sigaction(SIGALRM, &sa, NULL);
25     sigaction(SIGCHLD, &sa, NULL);
26
27     for (uint i = 2; i < argc; i++){
28         int ret = fork();
29         alarm(atoi(argv[1]));
30         if (ret == 0){
31             execlp(argv[i], argv[i], NULL);
32         }
33     }
34 }
```

figura 2 Código del ejercicio de signals

Queremos controlar la ejecución, duración y finalización de estos programas mediante signals. Contesta las siguientes preguntas.

1) ***(1 puntos)*** Si queremos usar los signals SIGALRM, SIGUSR1 y SIGCHLD para controlar la duración, ejecución y finalización de los procesos: Indica qué llamada(s) a sistema deberíamos añadir y en qué línea(s) de código para asegurar que:
   - SIGALRM está desbloqueado en el padre y bloqueado en los hijos
   - SIGUSR1 está bloqueado en el padre y desbloqueado en los hijos
   - SIGCHLD está desbloqueado en el padre y bloqueado en los hijos

   RESPUESTA: Al gestionar las señales, lo ideal es bloquearlas desde el principio y desbloquearlas cuando estemos listos para procesarlas. La llamada para modificar la máscara de señales es `sigprocmask`. Como no podemos garantizar el estado de las señales al inicio del programa, debemos bloquear/desbloquear explícitamente aquellas que lo requieran.

   Dado que la instrucción no especifica el estado de las demás señales, podemos usar `SIG_BLOCK` y `SIG_UNBLOCK`, pero consideramos incorrecto usar `SIG_SETMASK`. Una posible solución sería:

```
/*Añadir al principio de la página: por ejemplo, después de la línea 26*/
sigset_t m_pare, m_fills;
sigemptyset(&m_pare); sigaddset(&m_pare, SIGALRM);sigaddset(&m_pare, SIGCHLD);
sigprocmask(SIG_UNBLOCK, &m_pare, NULL);
sigemptyset(&m_pare); sigaddset(&m_pare, SIGUSR1);
sigprocmask(SIG_BLOCK, &m_pare, NULL);
/* Añadir al código de los hijos, después de la línea 30 */
sigemptyset(&m_fills); sigaddset(&m_fills, SIGALRM);sigaddset(&m_fills, SIGCHLD);
sigprocmask(SIG_BLOCK, &m_pare, NULL);
sigemptyset(&m_fills); sigaddset(&m_fills, SIGUSR1);
sigprocmask(SIG_UNBLOCK, &m_fills, NULL);
```

2) ***(0,5 puntos)*** Si el proceso padre recibiera un SIGALRM o un SIGCHLD, ¿Qué signal(s) estarían bloqueados/desbloqueados durante la ejecución de la función f_handler()?

   RESPUESTA: Como la máscara indicada para la señal en ambos casos está vacía, solo se bloqueará la señal que se esté procesando, es decir, si estamos procesando una SIGALRM, se bloqueará SIGALRM y si estamos procesando SIGCHLD y SIGCHLD.

3) ***(1,25 puntos)*** Indica cómo sería el contenido de la función f_handler para implementar los requisitos siguientes y qué modificaciones habría que hacer al programa principal para poder implementar la gestión de SIGALRM y SIGCHLD:
   - ***(0,5 puntos)*** Si estamos gestionando SIGALRM, queremos enviar un SIGKILL a todos los procesos hijos (no hace falta comprobar si están vivos).

   RESPUESTA: En este caso, una posible solución sería definir un vector de enteros para almacenar los PID y, mediante la llamada a kill, enviar una señal SIGKILL al handler. Podemos asumir que tenemos un máximo de procesos (por ejemplo, 10). Modificar el código principal para almacenar los PID (los PID deben ser una variable global). También se debe llevar un registro del total de procesos.

```
int pids[10], fills;
fills = argc -2;
```

   En la creación....

```
pids[i-2] = fork();
```

y en el handler

```
for (uint i = 0; i < fills; i ++) kill(pids[i], SIGKILL);
```

- ***(0,75 puntos)*** Si estamos gestionando el SIGCHLD, queremos imprimir el tiempo que queda hasta el timeout, fin de alarma, pero no cancelar la alarma y, además, queremos recoger el estado de finalización del/los procesos que hayan acabado e imprimir su PID

RESPUESTA: para mantener el tiempo pendiente de la alarma, podemos usar la misma llamada al sistema de alarma. Como no queremos cancelarla, tenemos que volver a llamar con el tiempo restante: por ejemplo

```
int remaining= alarm(0);
alarm(remaining)
sprintf(buffer,"Ens quedaven %d segonds\n", remaining);
write(1,buffer, strlen(buffer));
```

Para controlar la finalización de los archivos, usamos waitpid. Debemos tener en cuenta que puede haber más de un proceso que finalice, y para usarlo tendríamos que crear un bucle. Considerando lo que nos piden más adelante, la solución correcta sería usar WNOHANG. En general, bloquear un proceso en un handler de un signal no es correcto pero en este caso además, podría provocar que no se crearan todos los procesos. Una solución seria.

```
int ret,status;
while((ret=waitpid(-1, &status,WNOHANG) > 0)){
    if (WIFEXITED(status)){
       sprintf(buffer,"El procés %d acaba amb exit %d\n", ret, WEXITSTATUS(status));
  }else{
      sprintf(buffer,"El procés %d acaba amb exit %d\n", ret, WTERMSIG(status));
  }
  write(1,buffer,strlen(buffer));
}
```

4) ***(1,25 puntos)*** Queremos limitar el tiempo de ejecución de cada uno de los procesos hijos al número de segundos recibido en atoi(argv[1]). Se nos ocurren estas tres opciones:
   a. ***(0,75 puntos)***¿Alguna de ellas garantiza que el proceso padre reciba el SIGALRM cuando cada uno de los procesos hijos lleve en ejecución los atoi(argv[1]) segundos?

   b. ***(0,5 puntos)*** Comenta para cada opción si padre e hijos tendrán o no programada la alarma.

opción 1

```
for (uint i = 2; i < argc; i++){
  alarm(atoi(argv[1]));
  ret = fork();
```

RESPUESTA: No garantiza que todos tengan el mismo tiempo de espera, ya que la alarma se redefine en cada iteración. Solo se programará la que se definió al crear el último proceso, y el primero podría tener más tiempo. Solo la tendrá definida el padre, ya que no se hereda.

opción 2

```
alarm(atoi(argv[1]));
for (uint i = 2; i < argc; i++){
```

RESPUESTA: No garantiza que todos tengan el mismo tiempo de espera, ya que se define la alarma solo una vez; por lo tanto, a diferencia del caso anterior, los últimos hijos podrían tener menos tiempo. Solo él padre tendrá definida la alarma, ya que no se hereda.

opción 3

```
for (uint i = 2; i < argc; i++){
 int ret = fork();
 alarm(atoi(argv[1]));
```

RESPUESTA: No garantiza que todos los procesos tengan el mismo tiempo de "vida", ya que la alarma se redefine en cada iteración; solo se programará la que se definió al crear el último hijo, y por lo tanto el primero podría tener más tiempo. Aunque los hijos también configuren su alarma, en principio la tienen bloqueada.

5) ***(1 puntos)*** Considera el código con las modificaciones que hayas propuesto en la pregunta 3, haz una propuesta para evitar que el proceso inicial termine antes que sus hijos. Propón una solución lo más eficiente posible en cuanto a consumo de CPU. Indica qué código añadirías y en qué líneas. Restricción: La gestión de la finalización de procesos solo se puede hacer en la atención del SIGCHLD.

RESPUESTA: Para evitar que parezca que su ejecución ha finalizado y mantener la restricción de que la gestión de la finalización de los procesos solo se realiza con SIGCHLD, debemos usar `sigsuspend` después del bucle de creación de procesos. Hay que tener en cuenta que, aunque hayamos configurado una espera bloqueante para el handler de SIGCHLD, también debemos realizar alguna acción, ya que solo al salir del bucle finalizará su ejecución. Por lo tanto, haremos lo siguiente:

En el handler podemos controlar el número total de procesos hijos muertos y, cuando no queden más, finalizar la ejecución, por ejemplo:

```
while((ret=waitpid())>0){
 fills_remains--;
 if (fills_remains==0) exit(0);
```

En main:

```
sigfillset(&m_pare);
sigdelset(&m_pare, SIGALRM);sigdelset(&m_pare,SIGCHLD);
while(sigsuspend(&m_pare));
```